MICROPROCESSORS & MICROCONTROLLER

8085 INSTRUCTION SET

INSTRUCTION SET - INTRODUCTION

- An instruction is a binary pattern designed inside a microprocessor to perform a specific function.
- 8085 has 74 instructions.
- Each instruction is represented by an 8-bit binary value.

INSTRUCTION SET GROUPS

- Data Transfer Instructions
- Arithmetic Instructions
- Logical Instructions
- Branch Instructions
- I/O and Machine control Instructions

DATA TRANSFER INSTRUCTIONS

LDA 16-bit address

- The contents of a memory location, specified by a 16-bit address in the operand, are copied to the accumulator (A).
- The contents of the source are not altered.
- Example: LDA 2000H

STA 16-bit address

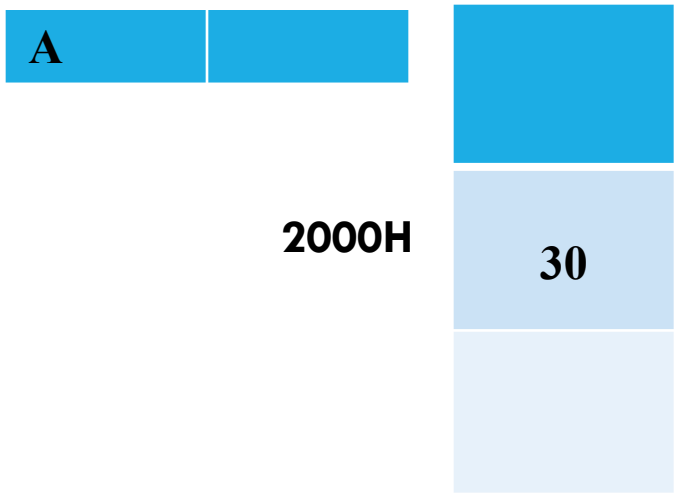
- The contents of accumulator are copied into the memory location i.e. address specified by the operand in the instruction.

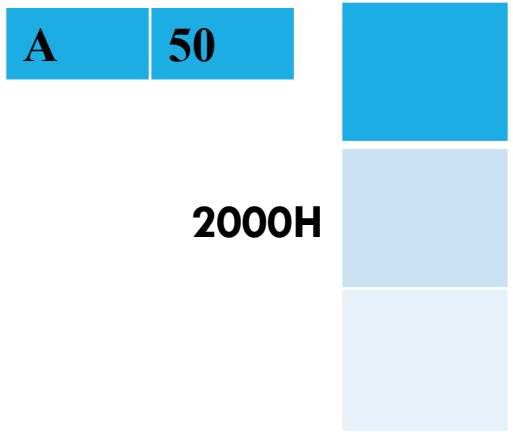
Example: STA 2000 H

LDA 2000H

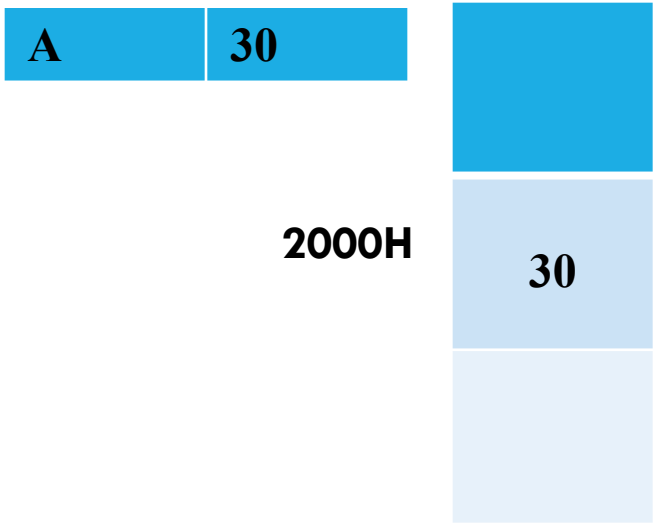
STA 2000H

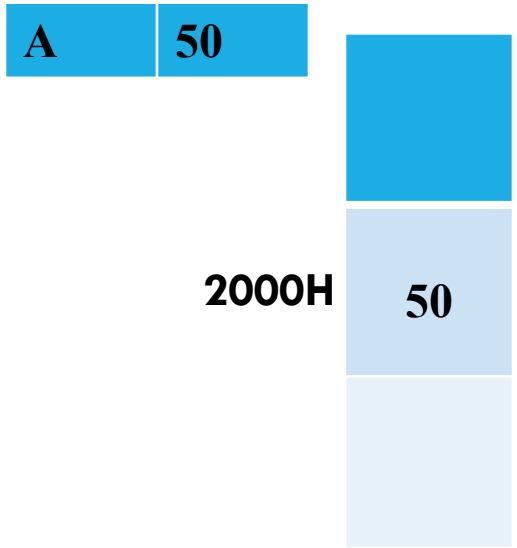
BEFORE EXECUTION





AFTER EXECUTION





DATA TRANSFER INSTRUCTIONS

- Copy the data from the source to a destination without modifying the content of the source.

```
MOV Rd, Rs

MOV M, Rs

MOV Rd, M

MVI  M,#VALUE

MVI  R,#VALUE
```

A	20	B	

MOV B,A

A	20	B	20

A		F	
B	30	C	
D		E	
H	20	L	50

MOV M,B

A		F	
B	30	C	
D		E	
H	20	L	50

2050

30

A		F	
B		C	
D		E	
H	20	L	50

40

MOV C,M

A		F	
B		C	40
D		E	
H	20	L	50

2050

40

A		F	
B		C	
D		E	
H		L	

MVI B,60H

A		F	
B	60	C	
D		E	
H		L	

204FH	
HL=2050H	
2051H	

MVI M,40H

204FH	
HL=2050H	40
2051H	

DATA TRANSFER INSTRUCTIONS

LDAX Register Pair

- Load accumulator (A) with the contents of memory location whose address is specified by BC or DE or register pair.
- The contents of either the register pair or the memory location are not altered.
- Example: LDAX D

STAX Register Pair

- Store the contents of accumulator (A) into the memory location whose address is specified by BC Or DE register pair.
- Example: STAX B

LDAX D

STAX B

BEFORE EXECUTION

A		F	
B		C	
D	20	E	30

2030H

80

A	50	F	
B	10	C	20
D		E	

1020H

AFTER EXECUTION

A	80	F	
B		C	
D	20	E	30

2030H

80

A	50	F	
B	10	C	20
D		E	

1020H

50

DATA TRANSFER INSTRUCTIONS

LHLD 16-bit address

- Copy the content of memory location into H-L register.
- The contents of memory location specified by the 16-bit address are copied into register L.
- The contents of next memory location are copied into the register H.
- Example : LHLD 3527 H

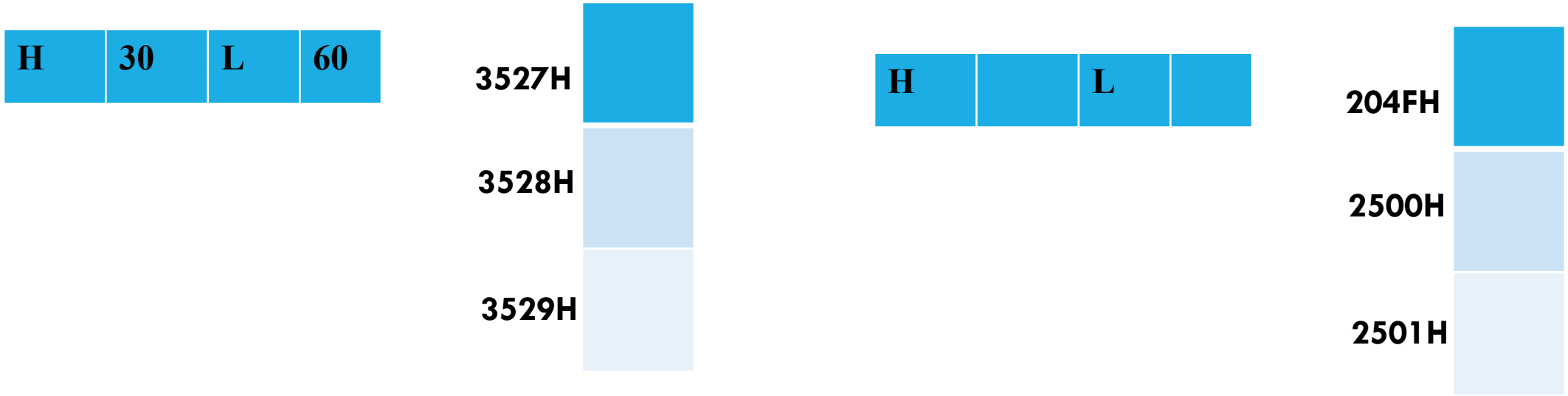
SHLD 16-bit address

- Store H-L register pair in memory.
- The contents of register L are stored into memory location specified by the 16-bit address.
- The contents of register H are stored into the next memory location.
- Example: SHLD 2500 H

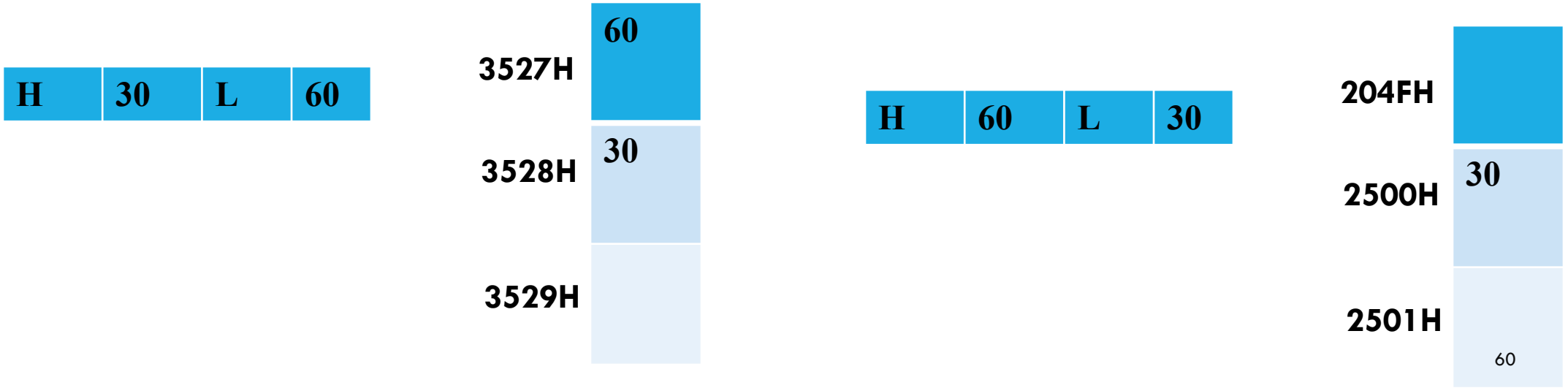
LHLD 3527 H

SHLD 2500 H

BEFORE EXECUTION



AFTER EXECUTION



DATA TRANSFER INSTRUCTIONS

XCHG

- The contents of register H are exchanged with the contents of register D.
- The contents of register L are exchanged with the contents of register E.
- Example: XCHG

BEFORE EXECUTION

D	20	E	40
H	70	L	80

AFTER EXECUTION

D	70	E	80
H	20	L	40

DATA TRANSFER INSTRUCTIONS

SPHL

- Move data from H-L pair to the Stack Pointer (SP)

BEFORE EXECUTION

SP			
H	25	L	00

SPHL

AFTER EXECUTION

SP	2500		
H		L	

DATA TRANSFER INSTRUCTIONS

XTHL

- Exchange H–L with top of stack
- The contents of L register are exchanged with the location pointed out by the contents of the SP.
- The contents of H register are exchanged with the next location (SP + 1).

L=SP

H=(SP+1)

BEFORE EXECUTION

SP	2700		
H	30	L	40

2700H	50
2701H	60
2702H	

XTHL

AFTER EXECUTION

SP	2700		
H	60	L	50

2700H	40
2701H	30
2702H	

DATA TRANSFER INSTRUCTIONS

PCHL

- Load program counter with H-L contents
- The contents of registers H and L are copied into the program counter (PC).
- The contents of H are placed as the high-order byte and the contents of L as the low-order byte.

BEFORE EXECUTION

PC			
H	60	L	00

PCHL

AFTER EXECUTION

PC	6000		
H	60	L	00

DATA TRANSFER INSTRUCTIONS

Opcode	Operand	Description
PUSH	Reg. pair	Push register pair onto stack

- The contents of register pair are copied onto stack.
- SP is decremented and the contents of high-order registers (B, D, H, A) are copied into stack.
- SP is again decremented and the contents of low-order registers (C, E, L, Flags) are copied into stack.
- **Example:** PUSH B

DATA TRANSFER INSTRUCTIONS

Opcode	Operand	Description
POP	Reg. pair	Pop stack to register pair

- The contents of top of stack are copied into register pair.
- The contents of location pointed out by SP are copied to the low-order register (C, E, L, Flags).
- SP is incremented and the contents of location are copied to the high-order register (B, D, H, A).
- **Example:** POP H

I/O INSTRUCTIONS

IN 8-bit port address

- Copy data to accumulator from a port with 8-bit address.
- The contents of I/O port are copied into accumulator.
- Example: IN 80 H

BEFORE EXECUTION

IN 80H

AFTER EXECUTION

PORT 80H

	10

A	

PORT 80H

	10

A	10

I/O INSTRUCTIONS

OUT 8-bit port address

- Copy data from accumulator to a port with 8-bit address
- The contents of accumulator are copied into the I/O port.
- Example: OUT 50 H

BEFORE EXECUTION

AFTER EXECUTION

ARITHMETIC INSTRUCTIONS

These instructions perform the operations like:

- Addition
- Subtraction
- Increment
- Decrement

ARITHMETIC INSTRUCTIONS

ADD R

ADD M

- The contents of register or memory are added to the contents of accumulator.
- The result is stored in accumulator.
- If the operand is memory location, its address is specified by H-L pair.
- Example: ADD C or ADD M

BEFORE EXECUTION

A	20		
B		C	30
D		E	
H		L	

ADD C
A=A+R

AFTER EXECUTION

A	50		
B		C	30
D		E	
H		L	

BEFORE EXECUTION

A	20		
B		C	
D		E	
H	20	L	50

10

ADD M
A=A+M

AFTER EXECUTION

A	30		
B		C	
D		E	
H	20	L	50

10

ARITHMETIC INSTRUCTIONS

ADC R

ADC M

- The contents of register or memory and Carry Flag (CY) are added to the contents of accumulator.
- The result is stored in accumulator.
- If the operand is memory location, its address is specified by H-L pair. All flags are modified to reflect the result of the addition.

Example: ADC C or ADC M

BEFORE EXECUTION

CY	1

A	50

B		C	20
D		E	
H		L	

ADC C
A=A+R+C

AFTER EXECUTION

CY	0

A	71

B		C	20
D		E	
H		L	

BEFORE EXECUTION

CY	1

A	20

H	20	L	50

2050H

30

ADC M
A=A+M+CY

AFTER EXECUTION

CY	0

A	51

H	20	L	50

2050H

30

ARITHMETIC INSTRUCTIONS

ADI 8-bit data

- The 8-bit data is added to the contents of accumulator.
- The result is stored in accumulator.
- Example: ADI 10 H

ACI 8-bit data

- The 8-bit data and the Carry Flag (CY) are added to the contents of accumulator.
- The result is stored in accumulator.
- Example: ACI 20 H

BEFORE EXECUTION

AFTER EXECUTION

A	50

ADI 10H

A=A+DATA(8)

A	60

CY	1
A	30

ACI 20H

A=A+DATA (8)+CY

CY	0
A	51

ARITHMETIC INSTRUCTIONS

DAD Register pair

- The 16-bit contents of the register pair are added to the contents of H-L pair.
- The result is stored in H-L pair.
- If the result is larger than 16 bits, then CY is set.
- Example: DAD D

BEFORE EXECUTION

CY	0

SP			
B		C	
D	10	E	20
H	20	L	50

DAD D

HL=HL+R

AFTER EXECUTION

CY	0

SP			
B		C	
D	10	E	20
H	30	L	70

ARITHMETIC INSTRUCTIONS - SUBTRACTION

INSTRUCTION	OPERATIONS
SUB R	A=A-R
SUB M	A=A-M
SBB R	A=A-R-CY
SBB M	A=A-M-CY
SUI 8-bit data	A=A-DATA(8)
SBI 8-bit data	A=A-DATA(8)-CY

ARITHMETIC INSTRUCTIONS

INR R

INR M

- The contents of register or memory location are incremented by 1.
- The result is stored in the same place.
- If the operand is a memory location, its address is specified by the contents of H-L pair.
- Example: INR B or INR M

BEFORE EXECUTION

A			
B	10	C	
D		E	
H		L	

INR B
R=R+1

AFTER EXECUTION

A			
B	11	C	
D		E	
H		L	

BEFORE EXECUTION

H	20	L	50

2050H

30

INR M
M=M+1

AFTER EXECUTION

H	20	L	50

2050H

31

ARITHMETIC INSTRUCTIONS

INX Rp

- The contents of register pair are incremented by 1.
- The result is stored in the same place.
- Example: INX H

BEFORE EXECUTION

SP			
B		C	
D		E	
H	10	L	20

INX H

RP=RP+1

AFTER EXECUTION

SP			
B		C	
D		E	
H	10	L	21

ARITHMETIC INSTRUCTIONS - DECREMENT

INSTRUCTIONS	OPERATIONS
DCR R	R=R-1
DCR M	M=M-1
DCX RP	RP=RP-1

ARITHMETIC INSTRUCTIONS

Also known as Decimal Adjust Accumulator.

This instruction is executed after ADD instruction.

(1). IF lower bits AL>09 then,

Bits of AL must be incremented by 06 (i.e. AL+0110).

(2). IF higher bits AH>09 then,

Bits of AH must be incremented by 06 (i.e. AH+0110).

E.g. DAA;

A= 39 C=12
A + C = A

(1). FOR AL>09H

A	4	B
	Hb	Lb

BEFORE EXECUTION

A	5	1
	Hb	Lb

AFTER EXECUTION

Hb=Higher bits,
Lb=Lower bits.

(1). FOR AH>09H

A	9	D
	Hb	Lb

BEFORE EXECUTION

A= 96 C=07
A + C = A

A	0	3
	Hb	Lb

AFTER EXECUTION

LOGICAL INSTRUCTIONS

These instructions perform the operations like:

- AND
- OR
- XOR
- COMPARE

LOGICAL INSTRUCTIONS – “AND”

ANA R

ANA M

- AND specified data in register or memory with accumulator.
- Store the result in accumulator (A).
- Example: ANA B, ANA M

BEFORE EXECUTION

CY		AC	

A	B3

AFTER EXECUTION

CY	0	AC	1

A	33

ANI 3FH
A=A and DATA(8)

1011 0011=B3H
0011 1111=3FH

0011 0011=33H

BEFORE EXECUTION

CY		AC	

A	AA

B	0F	C	
D		E	
H		L	

1010 1010=AAH
0000 1111=0FH

0000 1010=0AH

ANA B
A=A and R

AFTER EXECUTION

CY	0	AC	1

A	0A

B	0F	C	
D		E	
H		L	

BEFORE EXECUTION

CY		AC	

A	55

2050H

H	20	L	50

B3

0101 0101=55H
1011 0011=B3H

0001 0001=11H

ANA M
A=A and M

AFTER EXECUTION

CY	0	AC	1

A	11

2050H

H	20	L	50

B3

LOGICAL INSTRUCTIONS – "AND"

ANI 8-bit data

- AND 8-bit data with accumulator (A).
- Store the result in accumulator (A)
- Example: ANI 3FH

BEFORE EXECUTION

CY		AC	

A	B3

1011 0011=B3H
0011 1111=3FH

0011 0011=33H

ANI 3FH
A=A and DATA(8)

AFTER EXECUTION

CY	0	AC	1

A	33

LOGICAL INSTRUCTIONS

INSTRUCTION	OPERATIONS
XRA R	A=A xor R
XRA M	A=A xor M
XRI 8-bit data	A=A xor DATA(8)
ORA R	A=A or R
ORA M	A=A or M
ORI 8-bit data	A=A or DATA(8)

LOGICAL INSTRUCTIONS

CMP Register

CMP M

- Compare specified data in register or memory with accumulator (A).
- Example: CMP D or CMP M

BEFORE EXECUTION

CY		Z	

A	57

B	10	C	
D	62	E	
H		L	

A>R: CY=0,Z=0
A=R: CY=0,Z=1
A<R: CY=1,Z=0

CMP D
A - R

AFTER EXECUTION

CY	1	Z	0

A	57

B		C	
D	62	E	
H		L	

BEFORE EXECUTION

CY		Z	

A	B8

H	20	L	50

B8

2050H

A>M: CY=0,Z=0
A=M: CY=0,Z=1
A<M: CY=1,Z=0

CMP M
A-M

AFTER EXECUTION

CY	0	Z	1

A	B8

H	20	L	50

B8

2050H

LOGICAL INSTRUCTIONS

CPI 8-bit data

- Compare 8-bit immediate data with accumulator (A).
- Example: CPI 30H

BEFORE EXECUTION

CY		Z	

A	BA

CPI 30H
A-DATA

AFTER EXECUTION

CY	0	AC	0

A	BA

LOGICAL INSTRUCTIONS

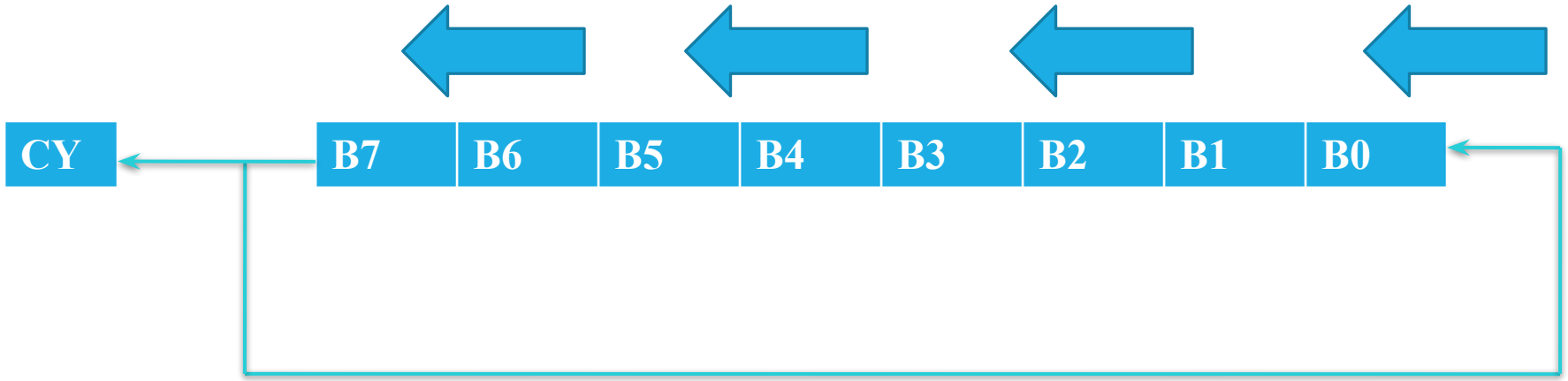
INSTRUCTION	OPERATIONS
STC	SET Carry flag (i.e) CY = 1
CMC	COMPLEMENT Carry Flag
CMA	COMPLEMENT each bit of accumulator

LOGICAL INSTRUCTIONS

RLC

- Rotate accumulator left
- Each binary bit of the accumulator is rotated left by one position.
- Bit D7 is placed in the position of D0 as well as in the Carry flag.
- CY is modified according to bit D7.
- Example: RLC.

BEFORE EXECUTION



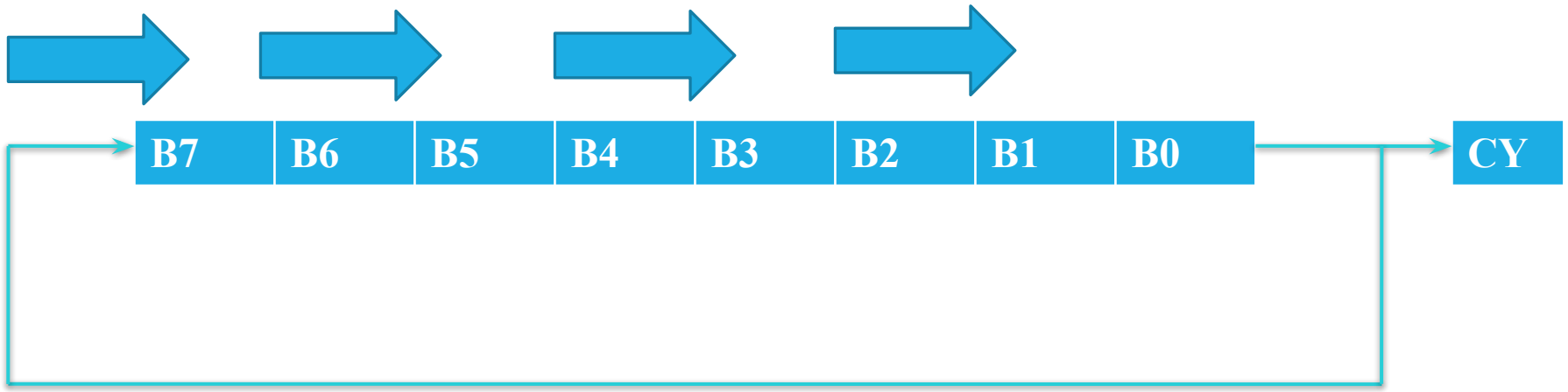
AFTER EXECUTION

LOGICAL INSTRUCTIONS

RRC

- Rotate accumulator right
- Each binary bit of the accumulator is rotated right by one position.
- Bit D0 is placed in the position of D7 as well as in the Carry flag.
- CY is modified according to bit D0.
- Example: RRC.

BEFORE EXECUTION



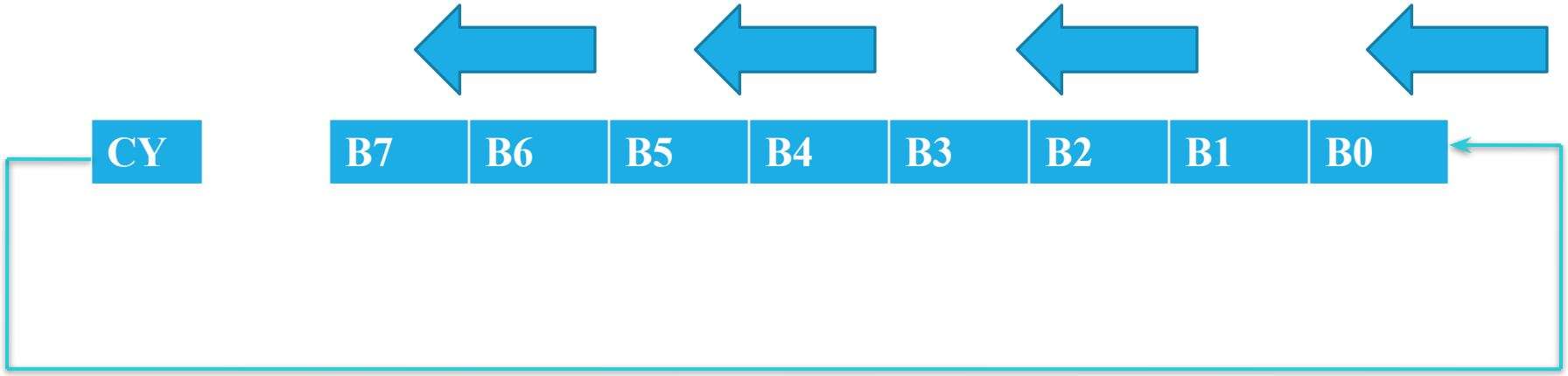
AFTER EXECUTION

LOGICAL INSTRUCTIONS

RAL

- Rotate accumulator left through carry
- Each binary bit of the accumulator is rotated left by one position through the Carry flag.
- Bit D7 is placed in the Carry flag, and the Carry flag is placed in the least significant position D0.
- CY is modified according to bit D7.
- Example: RAL

BEFORE EXECUTION



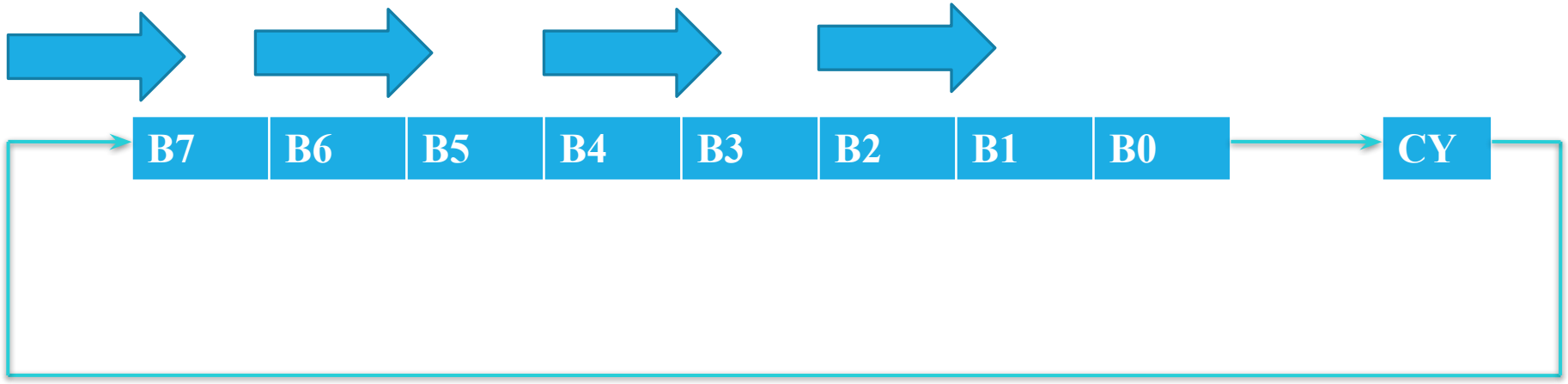
AFTER EXECUTION

LOGICAL INSTRUCTIONS

RAR

- Rotate accumulator right through carry
- Each binary bit of the accumulator is rotated left by one position through the Carry flag.
- Bit D7 is placed in the Carry flag, and the Carry flag is placed in the least significant position D0.
- CY is modified according to bit D7.
- Example: RAR

BEFORE EXECUTION



AFTER EXECUTION

CONCEPT OF SUBROUTINE

- In 8085 microprocessor a subroutine is a separate program written aside from main program ,this program is basically the program which requires to be executed several times in the main program.
- The microprocessor can call subroutine any time using CALL instruction. after the subroutine is executed the subroutine hands over the program to main program using RET instruction.

BRANCHING INSTRUCTIONS

The branch group instructions allows the microprocessor to change the sequence of program either conditionally or under certain test conditions. The group includes,

(1) Jump instructions,

(2) Call and Return instructions,

(3) Restart instructions

BRANCHING INSTRUCTIONS

JUMP ADDRESS

- Jump unconditionally to the address.
- The instruction loads the PC with the address given within the instruction and resumes the program execution from specified location.
- Example: JMP 200H

BEFORE EXECUTION

PC	

JMP 2000H

AFTER EXECUTION

PC	2000

CONDITIONAL JUMPS

Instruction Code	Description	Condition For Jump
JC	Jump on carry	**CY=1**
JNC	Jump on not carry	**CY=0**
JP	Jump on positive	**S=0**
JM	Jump on minus	**S=1**
JPE	Jump on parity even	**P=1**
JPO	Jump on parity odd	**P=0**
JZ	Jump on zero	**Z=1**
JNZ	Jump on not zero	**Z=0**

BRANCHING INSTRUCTIONS

CALL address

Call unconditionally a subroutine whose starting address given within the instruction and used to transfer program control to a subprogram or subroutine.

Example: CALL 2000H

CONDITIONAL CALLS

Instruction Code	Description	Condition for CALL
CC	Call on carry	CY=1
CNC	Call on not carry	CY=0
CP	Call on positive	S=0
CM	Call on minus	S=1
CPE	Call on parity even	P=1
CPO	Call on parity odd	P=0
CZ	Call on zero	Z=1
CNZ	Call on not zero	Z=0

BRANCHING INSTRUCTIONS

RET

Return from the subroutine unconditionally.

This instruction takes return address from the stack and loads the program counter with this address.

Example: RET

BEFORE EXECUTION

SP	27FD
PC	

27FDH	00
27FEH	62
27FFH	

AFTER EXECUTION

SP	27FF
PC	6200

27FDH	00
27FEH	62
27FFH	

BRANCHING INSTRUCTIONS

RST n

Restart n (0 to 7)

This instruction transfers the program control to a specific memory address. The processor multiplies the RST number by 8 to calculate the vector address.

Example: RST 6

BEFORE EXECUTION

AFTER EXECUTION

SP-1

SP	3000
PC	2000

2FFEH	
2FFFH	
3000H	

RST 6

SP	2999
PC	0030

2FFEH	01
2FFFH	20
3000H	

ADDRESS OF THE NEXT INSTRUCTION IS 2001H

CONTROL INSTRUCTIONS

- NOP
- HLT
- RIM
- SIM

CONTROL INSTRUCTIONS

NOP

- No operation
- No operation is performed.
- The instruction is fetched and decoded but no operation is executed.

- Example: NOP

CONTROL INSTRUCTIONS

HLT

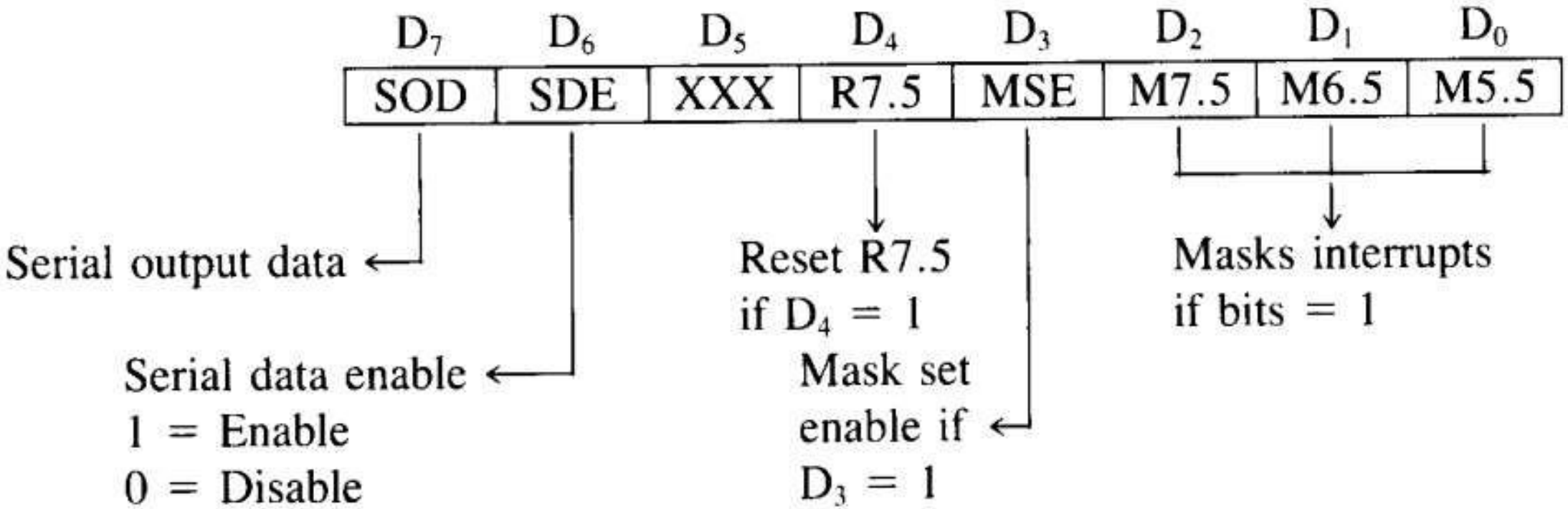
- Halt
- The CPU finishes executing the current instruction and halts any further execution.
- An interrupt or reset is necessary to exit from the halt state.
- Example: HLT

CONTROL INSTRUCTIONS

SIM

- Set Interrupt Mask
- This is a multipurpose instruction and used to implement the 8085 interrupts 7.5, 6.5, 5.5, and serial data output.
- The instruction interprets the accumulator contents as follows.
- Example: SIM

SIM INSTRUCTION

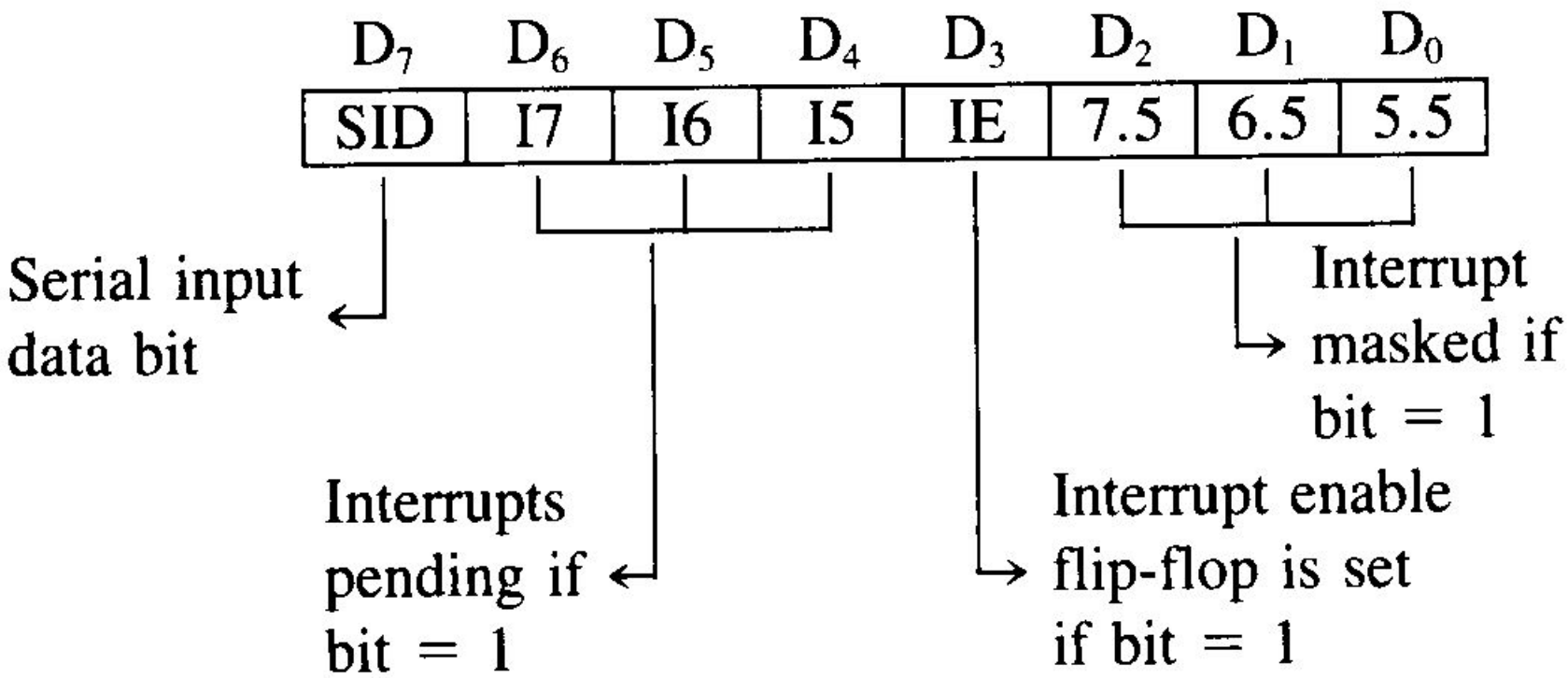


CONTROL INSTRUCTIONS

RIM

- Read Interrupt Mask
- This is a multipurpose instruction used to read the status of interrupts *7.5*, *6.5*, *5.5* and read serial data input bit.
- The instruction loads eight bits in the accumulator with the following interpretations.
- Example: RIM

RIM INSTRUCTION



Program to countdown

```
MVI D,0AH
MVI B,45H
MVI C,00H
L1: MOV A,D
STAX B
INX B
DCR D
JNZ L1
HLT
```

Program to UPcount

```
MVI D,0AH
MVI E,00H
MVI B,45H
MVI C,00H
L1: MOV A,E
STAX B
INX B
INR E
MOV A,E
CMP D
JNZ L1
HLT
```